

Efficiently Supporting Attribute-Based Access Control in Relational Databases

Gaurav Meena, Proteet Paul and Shamik Sural

Indian Institute of Technology Kharagpur, India

gdhawaniya@gmail.com, proteetpaul@gmail.com, shamik@cse.iitkgp.ac.in

Abstract—While Attribute-Based Access Control (ABAC) is increasingly becoming popular as a topic of research, it is yet to get traction in real applications. One of the reasons for this gap is that unlike Role-Based Access Control, which got support both from the software developers as well as the database community, ABAC is still not supported in relational databases. In this paper, we propose a comprehensive extension to SQL for attribute-based access control in relational databases covering all kinds of database objects like tables, views, stored procedures and triggers. The different types of ABAC attributes including subject, object as well as environmental attributes, are also supported in the proposed SQL extension. Further, we show how such an ABAC extension can be embedded in one of the most popular open source relational databases, namely MySQL. Towards this, we appropriately augment the source code of MySQL with an efficient method for ABAC policy enforcement. The ABAC augmented MySQL source code is being shared for reproducibility of our results, and also for any potential user to install this version and work with it. Findings from an extensive set of experiments establish the feasibility of our approach.

Index Terms—Relational databases, SQL, Access Control, ABAC, MySQL, Policy Enforcement

I. INTRODUCTION

Relational Database Management Systems (RDBMSs) constitute the primary data management technology used by all organizations today for a wide range of functions - from routine business operations to strategic decision making. Hence, the importance of security in such databases cannot be overemphasized [1]. Protection of data in a relational database is achieved through the interplay of several components. One of the primary enablers is its supported access control mechanism, which is responsible for ensuring both data confidentiality and integrity. Whenever a user tries to access a piece of data, the user's rights are verified against a pre-specified set of authorizations that indicate whether the user is allowed to perform the intended action on the object, and are governed by the organization's access control policy.

Relational databases currently support specification of Discretionary Access Control (DAC) policies using SQL commands like GRANT and REVOKE. Role-Based Access Control (RBAC) is also enforceable through the use of the CREATE ROLE command followed by GRANT. However, there is no comprehensive support yet for Attribute-Based Access Control (ABAC) in RDBMSs even though some work on supporting ABAC in NoSQL has been done [2] [3]. In a recent work, we briefly introduced an initial direction for attribute-

based access control in relational databases [4]. Named ASQL (ABAC Support for SQL), a subset of database objects were brought under ABAC by partially extending SQL. However, it has support only for tables and not any other type of database object. It also cannot handle resource attributes or environmental attributes - one of the distinguishing features of ABAC that makes it more expressive compared to DAC and RBAC. There is also no efficient method for access mediation.

In this context, we identify the following research questions.

- RQ1: Since access to relational databases can take place only using SQL statements, how to extend it for supporting all the ABAC components?
- RQ2: In an RDBMS, how to support ABAC-extended SQL efficiently? It has three sub-questions.
 - RQ2A: Which additional data structures are required to represent the various ABAC components?
 - RQ2B: How to interpret the additional SQL syntax for ABAC as identified in RQ1?
 - RQ2C: How to efficiently enforce the access requests using ABAC once the components have been set up?

This paper attempts to address the above research questions by making the following unique contributions.

- We propose ASQLPLUS - A comprehensive support for ABAC in SQL that covers all the different types of database objects. It also includes options to specify resource and environmental attributes. The syntax for ASQLPLUS is kept close to that of the existing access control policy specification syntax in SQL for DAC and RBAC.
- Viability of specifying ABAC policies in ASQLPLUS is demonstrated by appropriately updating the source code and system database tables of MySQL - one of the most popular open source RDBMSs.
- Besides specifying ABAC policies, we also show how access requests can be efficiently resolved in MySQL by parsing the ASQLPLUS commands and evaluating the ABAC rules using PolTree - a recently proposed data structure for ABAC [5].
- We package the ABAC specification and enforcement support in MySQL (and refer to it as AEMS - ABAC Enhanced MySQL) as a complete installable component, and make it publicly available for use by the community, with an intention to foster further development.

The rest of the paper is organized as follows. Section II provides a brief overview of the preliminaries. Section III presents the syntax and semantics of ASQLPLUS in detail. Support for ASQLPLUS in MySQL is described in Section IV. Section V discusses the results of an extensive set of experiments carried out on an ASQLPLUS enhanced MySQL instance for different workloads. We review related work in Section VI, and finally conclude with future directions for research in Section VII.

II. PRELIMINARIES

This section introduces some of the basic concepts underlying the work presented later in the paper.

A. Attribute-Based Access Control

Access in attribute-based access control is determined by attributes, which are associated with the entities involved, namely, users, resources and environment. For example, consider a university management database where *students* and *teachers* are users while *course* table and *class taken* table are the resources. In this case, user attributes could be the *designation* as well as the *department* of a user. *Document Sensitivity* and *Document Type* are the resource attributes. *Time* and *Day of the Week* are some of the possible environment attributes. Each of these attributes takes certain values that are either continuous (e.g., time) or discrete (e.g., designation). Assignment of values to user and environment attributes is usually done by the system administrator while for resources, it is the responsibility of the respective data owners. An ABAC policy is a combination of attribute-value pairs to specify the authorized operations as a set of access rules. In its simplest form, a rule is a Boolean combination of conditions imposed on each attribute. Thus, the various components of ABAC can be represented using the following notations:

- *Users (U)*: Set of authorized users
- *Resources (R)*: Set of objects on which access is restricted
- *Environment (E)*: Set of environmental conditions
- *User Attributes (UA)*: Set of attributes which are characteristics of user
- *Resource Attributes (RA)*: Set of attributes which are characteristics of resources
- *Environment Attributes (EA)*: Set of attributes which are characteristics of the environment in which access requests are evaluated
- *Operations (OP)*: Valid operations that can be performed by user on the resources
- *Policy (P)*: Set of rules, each being a Boolean combination of conditions imposed on some attributes

B. Policy Tree (PolTree)

Once the components of an ABAC system have been defined, it can be deployed to respond to access requests. Every request is comprised of attribute value pairs describing the user, the resource, the present environmental conditions and the operation that the user wants to carry out on the resource. Based on the policy rules, a reference monitor

determines whether to grant or deny the requested access. In ABAC, resolution of an incoming access request is done by comparing the attributes of the request with the attributes contained in various rules mentioned in the policy defined by the administrator. Until all the rules are exhausted or an access granting rule is found, this search is continued. Such sequential searching of rules is evidently inefficient, especially when there is no rule that grants the desired access. In organizations with large number of rules, this kind of access request resolution significantly increases the overhead, adversely affecting the performance of ABAC.

To mitigate this challenge, a tree data structure called PolTree was proposed by Nath et al. [5]. It has two variants, namely, Binary PolTree (B-PolTree) and n-ary PolTree (N-PolTree). Since N-PolTree has been shown to be more efficient, we consider it in our work (PolTree will denote N-PolTree in the rest of the paper). N-PolTree stores an ABAC policy efficiently and significantly reduces the access request resolution time. Each non-leaf node of an N-PolTree contains an attribute name, the operations allowed till now, and has branches for each distinct value of the attribute present in the rules. On the other hand, each leaf node consists of the operations allowed if a path exists from the root to this leaf for the incoming request attributes.

III. SYNTAX AND SEMANTICS OF ASQLPLUS

In order to address the research question RQ1 identified in Section I, here we first present the complete syntax and semantics of our proposed ASQLPLUS for comprehensively supporting ABAC in SQL. It includes commands for creating attributes, their values, policies, as well assigning these to the users and resources. We then give an illustrative example to explain how ASQLPLUS can be used to specify ABAC policies.

A. ASQLPLUS Specification

The syntax of ASQLPLUS, which includes the additional constructs for ABAC in SQL, is as follows.

- `CREATE RULE rule_name FOR priv_type [, priv_type] ... OF USER ATTRIBUTE {user_attribute = value|ANY, user_attribute = value|ANY ...} [AND RESOURCE ATTRIBUTE {resource_attribute = value|ANY, resource_attribute = value | ANY ...}] AND ENVIRONMENT ATTRIBUTE weekday_value, daytime_value [ON database_name]`
- `DROP RULE rule_name`
- `CREATE USER ATTRIBUTE user_attribute_name`
- `CREATE RESOURCE ATTRIBUTE resource_attribute_name`
- `DROP USER ATTRIBUTE user_attribute_name`
- `DROP RESOURCE ATTRIBUTE resource_attribute_name`

- GRANT USER ATTRIBUTE user_attribute = value TO user [, user] ...
- GRANT RESOURCE ATTRIBUTE resource_attribute = value TO resource [, resource] ...
- REVOKE USER ATTRIBUTE user_attribute [= value | ALL] FROM user [, user] ...
- REVOKE RESOURCE ATTRIBUTE resource_attribute [= value | ALL] FROM resource [, resource] ...

The semantics for these statements are briefly explained below:

- CREATE RULE statement: Creates a rule in the ABAC policy with various privileges on various database objects based on requirements. For example, privileges like INSERT, DELETE, SELECT, DROP, EXECUTE, ALTER PROCEDURE, TRIGGER, UPDATE, CREATE TABLE, CREATE VIEW, CREATE FUNCTION, CREATE PROCEDURE and CREATE TRIGGER can be applied on columns, tables, views, procedures/function or triggers accordingly. Please note that rules with CREATE TABLE, CREATE VIEW, CREATE FUNCTION and CREATE PROCEDURE operations have on database_name part with resource attribute part absent from the syntax.
- DROP RULE statement: Deletes a rule from the policy.
- CREATE USER (RESOURCE) ATTRIBUTE statement: Allows creation of a new user (resource) attribute.
- DROP USER (RESOURCE) ATTRIBUTE statement: Deletes a user (resource) attribute from the system.
- GRANT USER (RESOURCE) ATTRIBUTE statement: Assigns an attribute value pair to the user (resource).
- REVOKE USER (RESOURCE) ATTRIBUTE statement: revokes the attribute value mapping with respect to a particular value mentioned in the statement.

In addition, ASQLPLUS supports two environmental attributes, namely, *day* and *time*. They have fixed semantics since giving access through them requires evaluation at run time, which is different for every such attribute. The first environmental attribute in ASQLPLUS syntax is *day* (weekday or weekend) and the second is *time* (day or night). It is thus observed that all the components of ABAC as identified in Sub-section II-A can be managed using the above-mentioned ASQLPLUS commands. Thus, research question RQ1 is addressed.

Usage of ASQLPLUS is shown below through an illustrative example.

B. Illustrative Example

In this sub-section, we introduce an example ABAC system which has the following components (with short forms to be used later shown in parenthesis):

- UA = {Department (*dept*), Clearance level (*level*)}
- RA = {Age (*age*), Security Level (*secllevel*)}

- EA = {Day (*day*), Time (*time*)}
- OP = {SELECT, INSERT, UPDATE, DELETE}

Rule	Dept	Level	Age	SecLevel	Day	Time	Operation
r ₁	HR	L2	NEW	LOW	WEEKDAY	DAY	TRIGGER
r ₂	IT	L2	–	–	WEEKEND	DAY	CREATE VIEW, CREATE TABLE
r ₃	HR	L1	OLD	LOW	WEEKDAY	NIGHT	INSERT, UPDATE
r ₄	IT	L1	NEW	HIGH	WEEKEND	NIGHT	DELETE, EXECUTE
r ₅	IT	L2	–	–	WEEKDAY	DAY	CREATE PROCEDURE

Table I: List of rules in the example policy

Assume that the attributes mentioned above can take up the following values:

- $V_{dept}^U = \{HR, IT\}$
- $V_{level}^U = \{L1, L2\}$
- $V_{age}^O = \{OLD, NEW\}$
- $V_{secllevel}^O = \{LOW, HIGH\}$
- $V_{day}^O = \{WEEKDAY, WEEKEND\}$
- $V_{time}^O = \{DAY, TIME\}$

The policy P of this illustrative ABAC system consists of five rules as shown in Table I. We also consider four users and three resources as components of our system, which are shown in Tables II and III, respectively. We assume that all the relations are part of a common database called *erp*.

Once the users and objects have been defined in the system, we can use the ASQLPLUS syntax defined in Sub-section III-A to assign user (resource) attributes and create policy rules. For instance, the following statements would be used to first create the user attributes.

- CREATE USER ATTRIBUTE Department;
- CREATE USER ATTRIBUTE Clearance_Level;

Once the attributes are created, appropriate values can be granted to the second user u_2 of Table II as follows:

- GRANT USER ATTRIBUTE Department = IT to u_2 ;
- GRANT USER ATTRIBUTE Clearance_Level = L2 to u_2 ;

For the third resource shown in Table III, the corresponding grant commands would be (assuming the appropriate CREATE RESOURCE ATTRIBUTE commands have already been executed):

- GRANT RESOURCE ATTRIBUTE Age = OLD to archives;
- GRANT RESOURCE ATTRIBUTE Security_Level = HIGH to archives;

Attribute values can similarly be granted to the rest of the users and objects as well.

Now we can define the rule r_3 shown in Table I as follows:
CREATE RULE r_3 FOR INSERT, UPDATE
OF USER ATTRIBUTE {Department = HR,
Clearance_Level = L1} AND RESOURCE
ATTRIBUTE {Age = OLD, Security_Level =
LOW} AND ENVIRONMENT ATTRIBUTE {weekend,
night};

User	Department	Clearance Level
u_1	HR	L1
u_2	IT	L2
u_3	HR	L2
u_4	IT	L1

Table II: List of users in the example ABAC system

Resource	Age	Security Level
emp_records	OLD	LOW
finance_books	NEW	HIGH
archives	OLD	HIGH

Table III: List of tables in the example ABAC system

This rule allows users from HR department and L1 clearance level (e.g., user u_1) to access tables with OLD and HIGH attribute values (e.g., *archives* table) on weekends during night-time to perform insert or update operations on them.

As another example, we can define the last rule shown in Table I as follows:

```
CREATE RULE r_5 FOR CREATE PROCEDURE
OF USER ATTRIBUTE {Department = IT,
Clearance_Level = L2} AND ENVIRONMENT
ATTRIBUTE {weekday, day} on hos_dat;
```

This rule would allow users with IT and L2 attribute values to have create procedure privilege on weekday and daytime on the hos_dat database.

While the syntax and semantics of ASQLPLUS is independent of any particular database management system, we next show how these can be supported in a popular RDBMS, namely MySQL, and attribute-based access control is enforced.

IV. ASQLPLUS SUPPORT IN MYSQL

Supporting ASQLPLUS in any RDBMS is an interesting problem that poses several research challenges as identified as RQ2A-RQ2C in the Introduction section. We discuss these in the context of MySQL. The issues and solutions would be similar, though not exactly the same, for other database management systems as well. We describe the challenges and present methods for handling them in the next few subsections.

A. MySQL System DB Extension for ASQLPLUS Support

MySQL currently supports DAC and RBAC through the use of a number of *System DB* tables. The main research challenges in supporting ASQLPLUS in MySQL include (i) Determining how the existing access control mechanism can be augmented with ABAC support with minimal changes (ii) Including views, stored procedures, functions and triggers as resources governed by ABAC policies and (iii) Adding support for environmental attributes.

It is imperative that the existing System DB tables would not suffice for implementing ABAC. DAC and RBAC policies are supported using a set of grant tables. These grant tables are to be modified indirectly for handling ABAC. The contents of the

additional tables needed for ASQLPLUS are briefly described below:

- **user_attributes**: Maintains the names of all possible user attributes
- **resource_attributes**: Maintains the names of all possible resource attributes.
- **user_attr_val**: Attribute-value pairs for each user. We assume that for a particular attribute, each user can be assigned only a single value.
- **resource_attr_val**: Attribute-value pairs for each resource. Once again, we assume that each resource can be assigned only a single value for a particular attribute.
- **policy**: Rule names and operations permitted by each rule. The operations allowed are SELECT, INSERT, UPDATE, DELETE, CREATE TABLE, CREATE PROCEDURE, CREATE VIEW, CREATE TRIGGER, EXECUTE, DROP and ALTER PROCEDURE.
- **policy_db**: Database name for policy rule in case of create privileges on tables, procedure, view or trigger.
- **policy_user_aval**: User attribute-value pairs for each policy rule.
- **policy_resource_aval**: Resource attribute-value pairs for each policy rule.

When all such System DB tables are appropriately populated, the ASQLPLUS enhanced MySQL instance becomes ready for ABAC policy enforcement. Thus, research question RQ2A is handled.

B. Processing of ASQLPLUS Commands in MySQL

Besides augmenting the system database for representing ABAC attributes and rules, it is also necessary for an RDBMS like MySQL to be able to interpret the augmented SQL syntax of ASQLPLUS as presented in Section III. This is the second research challenge involving identification of the appropriate source code level context of trapping SQL commands and extending the same without affecting the existing execution path. In order to address this challenge, the first step towards implementing ASQLPLUS is defining the syntax rules in the pre-existing MySQL parser.

The bison¹ rule for the modified create rule statement is shown in Listing 1. For each such statement, two classes are created: one for the parse tree node and the other for the command itself. The parse tree node class creates a corresponding `sql_cmd` object class providing its own implementation of the `sql_cmd::execute` function. The server daemon invokes this function, which in turn acts as an entry point to the processing of the command.

Processing of these commands is done by first checking sanity/validity of the same and then modifying the appropriate grant tables. In the context of the CREATE RULE statement, the following conditions are checked:

- Each user and object attribute is valid

¹<https://www.gnu.org/software/bison/>

Rule_name	Select_priv	Insert_priv	Update_priv	Delete_priv	Create_table_priv	Create_view_priv	Drop_priv	Create_proc_priv	Execute_priv	Alter_proc_priv	Trigger_priv	Weekday	Daytime	Db_level
r_1	N	N	N	N	N	N	N	N	N	N	N	N	Y	Y
r_2	N	N	N	N	Y	Y	N	N	N	N	N	N	Y	N
r_3	N	Y	Y	N	N	N	N	N	N	N	N	Y	N	N
r_4	N	N	N	Y	N	N	N	N	Y	N	N	N	N	N
r_5	N	N	N	N	N	N	N	Y	N	N	N	Y	Y	Y

Table IV: Content of Policy Table for our example

Rule_name	User_attrib_name	User_attrib_val
r_1	Dept	HR
r_1	Level	L2
r_2	Dept	IT
r_2	Level	L2
r_3	Dept	HR
r_3	Level	L1
r_4	Dept	IT
r_4	Level	L1
r_5	Dept	IT
r_5	Level	L2

Table V: Content of policy_user_aval for our example

Rule_name	Resource_attrib_name	Resource_attrib_val
r_3	Age	OLD
r_3	SecLevel	LOW
r_4	Age	NEW
r_4	SecLevel	HIGH

Table VI: Content of policy_resource_aval Table for our example

- Environment attribute values are valid
- Database exists if provided
- Policy contains no other rule having the same name

Next a few helper functions are used to modify the policy, policy_db (if required), policy_user_aval and policy_resource_aval tables in appropriate order so as to maintain referential integrity. The snippet shown in Listing 2 modifies the policy table. Its parameters include the rule name and the privileges. The third and the fourth parameter set the environment attribute values for the policy. The fifth parameter db_level_option checks whether the policy is at the db level or not. A sixth parameter called delete_option indicates whether the given rule should be deleted. This allows the same function to be utilized by both the CREATE RULE as well as the DELETE RULE statements.

The above-mentioned approach for updating the SQL parser to handle ASQLPLUS effectively addresses research question RQ2B identified in the Introduction section.

C. Enforcing Access Control in ASQLPLUS-Enhanced MySQL

In this sub-section, we first identify the main challenges in policy enforcement and then present a method for efficiently

```

create_rule_stmt:
    CREATE RULE_SYM ident FOR_SYM
    privilege_list OF_SYM USER
    ATTRIBUTE_SYM '('
    user_attribute_list ')'
    AND_SYM
    resource_attribute_list
    ENVIRONMENT_SYM
    ATTRIBUTE_SYM '(' ident ','
    ident ')' db_level
{
    $$ = NEW_PTN PT_create_rule(
        string($3.str), $5, $10,
        $13, string($17.str),
        string($19.str), string(
            $21.str));
}
;

```

Listing 1: Bison code for create rule statement

```

bool modify_rule_in_table(THD *thd, TABLE
    *table, string rule_name, int privs,
    int weekday, int daytime, bool
    db_level_option, bool delete_option);

```

Listing 2: function for modifying policy table

evaluating the policy rules for arriving at a decision whether the access should be granted or not.

1) *Access Control Enforcement Challenges*: After the policy has been defined in appropriate System DB tables and the parser can identify the ASQLPLUS commands being executed, the next research challenge is to enforce access control efficiently when a user attempts to execute any operation on a database object. It is more so when the existing source code that implements policy enforcement is to be modified without affecting its modularity and efficiency. While tables and views are handled in a similar manner, stored procedures and functions are controlled by different functional modules. Since creating trigger is a privilege which is associated with tables, a CREATE TRIGGER privilege was added as a possible operation on tables.

Another challenge is that for defining a policy, we need to mention the policy name, operations allowed, user attributes and resource attributes. Resource attributes are associated with the resources such as tables, views, triggers and procedures

(scope of resource attributes is limited to tables, views and other objects, not database itself). Thus, if the intent is to mention a policy that would give privilege to create a table or view, there is no obvious way of doing that. To address this challenge, we divide the rules into two types, one with create operations and others with operations like drop, execute, select, etc. Using such an approach, the database name could be included while mentioning create operation policies and store it in the policy_db table with all other information stored in the policy table.

The final challenge in policy enforcement is handling of environmental attributes. These are evaluated at run time and evaluation depends on the semantics of the specific attribute. Hence, a fixed set of environmental attributes cannot be handled. We currently work with two environmental attributes, namely Time and Day. The Time attribute can have two values, daytime and nighttime. It can be used to specify policies where access should only be provided at day or night depending on the need. The Day attribute currently can have two values, namely, weekday and weekend which, as the name suggests, can be used to create policies where access decision depends on weekdays or weekends. Evaluation of such time attributes can be done by fetching the current time using c++ libraries and comparing it with the value mentioned in the policy. The Day attribute can also be evaluated in a similar manner.

Note that, once environmental attributes are included in the ABAC rules, the access decisions cannot be statically determined using traditional methods like Access Control Lists (ACLs). Instead, we need to use the ABAC policies just the way they are, and not creating ACLs. When an access request comes, the policies are evaluated directly at run time, thus harnessing the full power of ABAC involving environmental attributes. In the next sub-section, we describe the steps.

2) *Efficient ASQLPLUS Support in MySQL*: As identified above, environmental attributes make ABAC policies more expressive. However, they also incur additional overhead during enforcement. Further, some applications may not need their use. Hence, initially we restrict ourselves to user and resource attributes only for implementing ABAC support for all database objects like views, stored procedures, functions and triggers. For this, we add in-memory data structures to the existing ones as shown in Listing 3. These data structures get updated whenever FLUSH PRIVILEGES command or any other ASQLPLUS commands are executed.

Information about individual grants is efficiently stored in an object of the pre-existing class TABLE_GRANT. These objects (views, stored procedures and functions) in turn are consolidated in the form of a hash map in the unordered_map variable *abac_rule_hash* or *abac_rule_proc_hash*. The hash key is generated by concatenating the user, database and table names, with the null character serving as a delimiter between each identifier. Update is carried out in a relatively straightforward manner using a brute force approach. That is, for every policy rule, we iterate through the list of users and tables in the system, creating an object of the TABLE_GRANT

```
extern unordered_map<std::string,
    GRANT_TABLE*> *abac_table_priv_hash;
extern unordered_map<std::string,
    ACL_USER_ABAC*> *acl_user_abac_hash
;
extern unordered_map<std::string,
    ABAC_OBJECT*> *abac_resource_hash;
extern unordered_map<std::string,
    ABAC_RULE*> *abac_rule_hash;
extern unordered_map<std::string,
    GRANT_NAME*> *
    abac_table_proc_priv_hash;
extern unordered_map<std::string,
    ABAC_RULE*> *abac_rule_proc_hash;
extern unordered_map<std::string,
    ABAC_RULE_DB*> *abac_rule_db_hash;
extern unordered_map<std::string,
    ABAC_RULE_DB*> *
    abac_rule_db_proc_hash;
extern unordered_set<std::string>
    *user_attribute_set;
extern unordered_set<std::string>
    *resource_attribute_set;
```

Listing 3: Data structures to store ABAC information

class for each such user and table combination that satisfies a given policy rule. To implement the solutions discussed above, we store the environmental attributes as well, while storing the policies in in-memory data structures.

In order to maintain parity with the existing MySQL architecture, we use the same set of in-memory data structures shown in Listing 3. These data structures store various components required for implementing the ABAC system, like policies, user to user attribute mappings, resource to resource attribute mappings, environmental attribute values, etc. These data structures are refreshed whenever the FLUSH PRIVILEGES command or any command that is part of the ASQLPLUS syntax is executed and new changes are made on the existing policies. Once set up, whenever an access request comes, we simply iterate through all the rules stored in the corresponding data structures at run time itself and access is granted accordingly.

This policy iteration process, however, has a significant overhead when there are a large number of rules in the system. Also, even if there is a rule that provides access, if it is located towards the end of the list, the search time would be significantly high. To tackle this efficiency problem, we use Policy Tree, which is an n-ary tree for representing and storing ABAC policies. As shown by Nath et al. [5], N-PolTree can significantly improve the performance of ABAC in an organization by facilitating fast resolution of access requests.

To store a MySQL database ABAC policy in a PolTree, we use additional data structure beyond the existing ones

used in the baseline method. Whenever these data structures are modified or refreshed, the ABAC tree is re-built with the update policy. There is one global tree which contains all the policies with resource attributes and also one tree for each database having policies with create operations. An N-PolTree is built using the *build_abac_pol_tree()* function, which follows the broad steps mentioned below [5]:

- Compute the attribute with the maximum entropy
- Determine the list of entities corresponding to the values of the selected attribute
- Recursively invoke the above two steps with reduced number of rules
- Create leaf node if no rules are left

When an access request arrives, the global tree and the db specific tree are traversed using the *check_access_abac()* function and access is granted accordingly. In N-PolTree, the maximum number of comparisons required to resolve an access request is $O(|A|)$, where $|A|$ is the number of attributes. Supporting N-PolTree in MySQL is a novel contribution of our work, which makes ABAC enforcement significantly efficient and effectively addresses research question RQ2C identified in the Introduction section.

D. Deployment of ASQLPLUS Enhanced MySQL

We have enhanced MySQL by appropriately updating its source code and system database tables for providing ASQLPLUS support including all the components mentioned in the last three sub-sections. This updated source has been made publicly available over GitHub for reproducibility of our results as well as for the research community to further enhance its functionality. On rebuilding the shared source, one can install AEMS - the ABAC Enhanced MySQL, and deploy databases with ABAC capabilities.

Our GitHub repository *mysql-server* [6] contains the complete source code and the dependencies for ASQLPLUS support on MySQL. The basic steps for installing AEMS from the repository are as follows:

- i Download the source code from the GitHub repository or clone the Git repository (in that case, skip the second step below).
- ii Extract the contents of the zip file to a directory on the target system.
- iii Install the necessary build dependencies for the target system. This may include packages such as build-essential, cmake, ncurses and libssl-dev. One will need to install specific versions of some of these dependencies.
- iv Configure the build directory using the *cmake* command. It will create a Makefile which would be used for compiling MySQL binaries. If there is any error, install the necessary build dependencies with the specific version mentioned.
- v Run the command *make* and *make install* to build and install AEMS. This will compile the binaries and libraries and install them on the system.
- vi Create the necessary additional system tables and directories for MySQL using the *mysql -initialize* command.

Parameter	Value
No. of UA	10
No. of RA	10
No. of EA	2
No. of AV (except env)	20
No. of tables	100

Table VII: Parameters for experiments in Figure 1

Parameter	Value
No. of Users	200
No. of RA	10
No. of EA	2
No. of AV (except env)	20
No. of tables	100

Table VIII: Parameters for experiments in Figure 2

- vii The final step is to start the MySQL daemon using the command *sudo systemctl start mysqld*.

The *aems_v1* branch in our repository contains ABAC support to all database objects without environmental attributes. It uses sequential search for access resolution. The *aems_v2* branch, on the other hand, has code for full ABAC support including environmental attributes along with the use of PolTree for access resolution.

V. EXPERIMENTAL RESULTS

We carried out several experiments with the ASQLPLUS enhanced MySQL instance. The server was set up on a 2.4GHz Intel processor having 8GB of RAM running Ubuntu 20.04, while the requests were locally routed to the server using Python MySQL connector. In the absence of publicly available ABAC datasets, we used a simulator to generate data. Each user and resource attribute was allowed to take up integer attribute values (AV) between 1 to 20, while environment attributes were allowed values 1 or 2. Since the execution time is not affected by the data type of the attributes, all were considered as integers.

We ran scripts to create the corresponding users and grant them the required attributes. The same was done for resources like tables. Rules were generated randomly. However, to ensure that access is granted for a user, say *u1*, and data could be fetched, rules which would give user *u1* SELECT and INSERT operations on all tables were added on top of the existing rules. This ensured that there exists at least one policy for each table which gives select permission to the user *u1* on that table. Five hundred rows were inserted into each table. We evaluated the performance of our approach by calculating the average execution time over five hundred different queries and each query was repeated 4 times. This was done to minimize the effect of other concurrently running system processes.

In the first experiment, we varied the number of rules in the ABAC policy keeping other parameters fixed as shown in Table VII. The corresponding values for execution time are plotted in Figure 1. From the figure, it is observed that the execution time is almost constant with increase in policy size

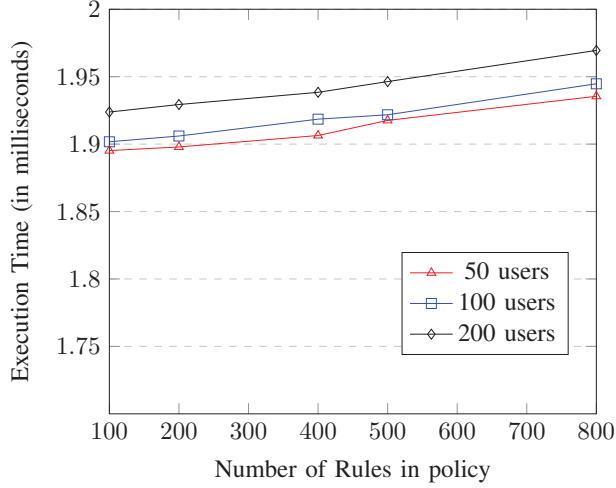


Figure 1: Query Execution Time vs. No. of Rules

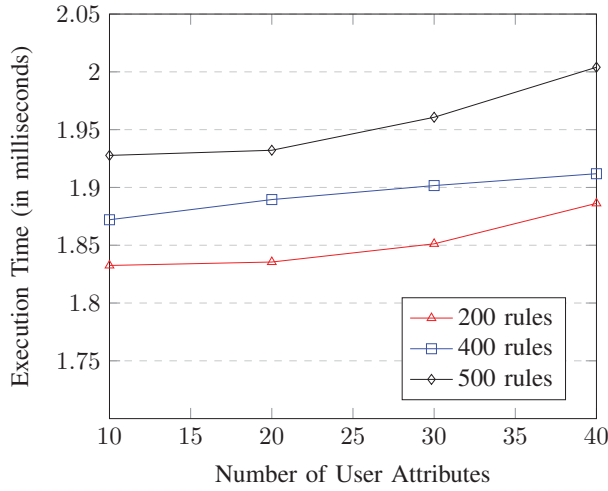


Figure 2: Query Execution Time vs. No. of User Attributes

as the number of attributes and their possible values are fixed. Also, varying the number of users does not have any significant impact on execution time.

In the second experiment, we study the variation in query execution time with number of user attributes keeping other parameters fixed as shown in Table VIII. The results are plotted in Figure 2. It is seen that as the number of user attributes is increased, the query execution time slowly increases. This can be explained from the fact that the traversal time for N-PolTree is proportional to the number of nodes, which in turn is equal to the number of attributes. We next vary the number of resource attributes keeping other parameters fixed as shown in Table IX. The results shown in Figure 3 are similar to those for varying the number of user attributes.

Finally, we compare sequential policy search execution time with N-PolTree keeping other parameters fixed as shown in Table X. The results in Figure 4 clearly indicate that the

Parameter	Value
No. of users	200
No. of UA	10
No. of EA	2
No. of AV(except env)	20
No. of tables	100

Table IX: Parameters for experiments in Figure 3

Parameter	Value
No. of users	200
No. of UA	10
No. of RA	10
No. of EA	2
No. of AV(except env)	20

Table X: Parameters for experiments in Figure 4

execution time for N-PolTree is significantly less compared to sequential search. The difference gets more pronounced with increase in number of rules in the policy, since PolTree is unaffected by such increase while sequential search is linearly affected by it.

Overall, the results of our experiments clearly indicate that the use of PolTree makes policy enforcement more efficient as compared to the sequential search method used in [4]. It is also unaffected by the policy size, number of users and number of resources. Further, since the query execution time is of the order of a few milliseconds on a client grade machine, it can be concluded that the overhead caused by the ABAC extension on the performance of MySQL is minimal. Thus, our work on ASQLPLUS enables one to harness the full power of ABAC in relational databases.

VI. RELATED WORK

Work on access control in relational databases started around the same time as the development of this technology. Griffiths and Wade [7] proposed one of the first access control models for RDBMSs in the context of System R. It was a DAC based model which allowed granting and revoking of privileges to users. Later, two major extensions to the System R authorization model were provided by Bertino et al. [8], the first being non-cascading revoke operation while the second dealt with negative authorizations. In contrast to DAC, Mandatory Access Control (MAC) models are more difficult to deploy in databases. This is primarily due to the fact that most database policies cannot be suitably mapped into a MAC model.

Reflective Database Access Control (RDBAC) [9] is yet another model which was developed to improve the expressiveness of database access control policies by representing policy as a database query itself. Role Based Access Control (RBAC) [10] has been deployed in various RDBMSs and is also included in SQL standard since RBAC can effectively represent a wide variety of organizational policies [11]. Among other models, Ranise and Truong [12] discuss the safety threats for Administrative Role-Based Access Control (ARBAC) policies

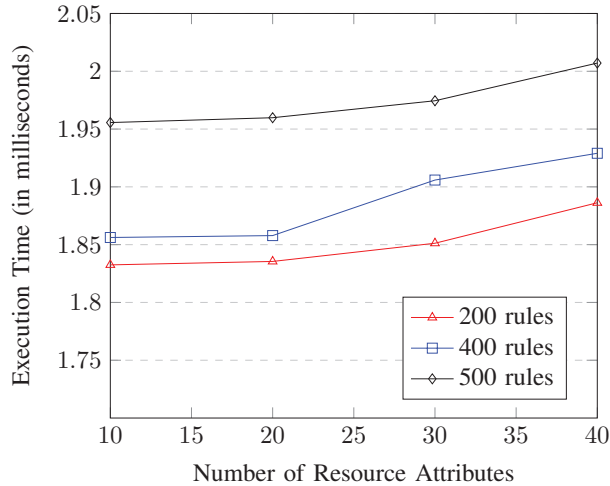


Figure 3: Query Execution Time vs. No. of Resource Attributes

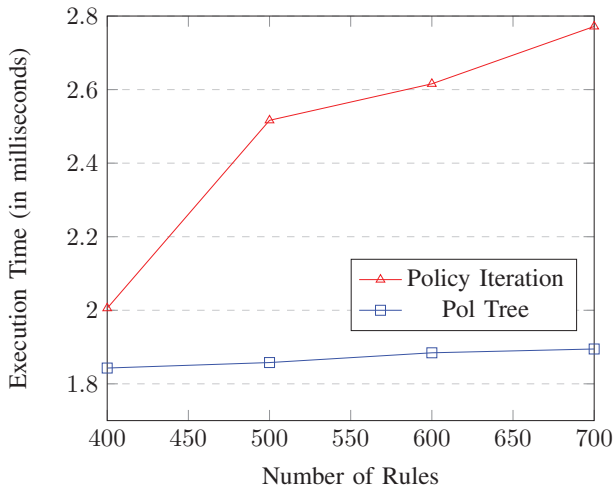


Figure 4: Query Execution Time vs. No. of Rules in policy

which occur because of sequence of actions leading to a possible security vulnerability. Shalabi and Gudes [13] propose a model for supporting RBAC using cryptography for Cassandra, a distributed NoSQL database.

In recent years, research on ABAC has focused on improving the efficiency and scalability of ABAC mechanisms, as well as developing standardized policy languages and interfaces. In 2014, NIST provided a standard definition for attribute based access control [14] as well as the functional components involved in it. They also discussed the various aspects of ABAC deployment in an organization with the aim to improve information sharing, while maintaining proper access over that information. Later, two other models for ABAC called Extensible Access Control Markup Language (XACML) and Next Generation Access Control (NGAC) [15] were proposed. A unified ABAC model was presented that

set connection between DAC, MAC, RBAC and the desired ABAC model. Furthermore, a Next Generation Database Access Control (NDAC) model [16] was suggested that enforces data level access control over database queries instead of traditional way of controlling access at application level or using middleware protection mechanisms.

Byun and Li proposed a non traditional access control approach [17] focusing on protecting privacy. It is primarily concerned with the purpose for which a certain data object could be accessed, a notion quite different from the traditional user-object-operation approach. Following this, a new approach called Purpose and Role Based Access Control (PuRBAC) [18] was presented for conducting execution of RDBMS queries based on purpose and role based privacy policies. An application based access control mechanism [19] was later presented for controlling access to remote RDBMSs ensuring data safety. This was implemented as a proxy program which regulates access from client applications to an RDBMS.

Some of the recent literature has addressed the problems of deploying ABAC in various systems including NoSQL databases. For example, Gupta et al. [3] deploy ABAC in MongoDB through a proxy that traps the incoming requests and converts them to appropriate MongoDB queries. In another related work [20], Jahid et al. propose an efficient mechanism for compiling XACML encoded ABAC policies into Access Control Lists (ACLs) that are implemented in databases. Their approach also includes a database-level mechanism to maintain the above mentioned ACLs. In contrast, Huang et al. propose an Attribute-Based Fine-Grained Access Control mechanism (AGAC) [21] to enable ABAC support in HBase, a big NoSQL database. Access control for object oriented databases has been explored in [22], which is based on controlling function evaluation.

There has also been significant progress in providing ABAC support for a wide variety of applications. For example, Hemdi and Deters [23] enable ABAC in IoT systems using a REST based protocol. Zhu et al. [24] present a dynamic ABAC model for cloud computing environments that supports fine-grained access control based on user attributes and resource attributes. Li and Yu [25] also propose an ABAC model for cloud computing that includes an efficient revocation mechanism or withdrawing access rights when needed. For online social networks, Swissa and Gudes [26] implemented the ABAC model in a system called ABAC-TG which considers user, object and social attributes for creating policies. There has also been some work [27] on attribute based messaging (ABM) which essentially allows to create a dynamic list of recipients based on their attributes, and send messages to them. Its authors were able to develop an efficient ABM implementation which could send messages with a few seconds of latency.

Notwithstanding the above different types of work on access control in RBMSs and ABAC implementation in various applications, There has not been any work done in implementing ABAC support for relational databases beyond an initial direction recently introduced by us [4]. Our current paper makes the work complete with full support for ABAC

in SQL Syntax for all kinds of database objects as well as an efficient method for access control enforcement in MySQL - one of the most popular open source RDBMSs.

VII. CONCLUSION

In a bid to support ABAC in RDBMSs, we first comprehensively specify an ABAC extension to SQL named as ASQLPLUS. We then show how in a real RDBMS, namely MySQL, ASQLPLUS can be implemented efficiently. This requires comprehensive understanding of the source code structure of MySQL and making appropriate changes to the code both for parsing incoming ASQLPLUS queries as well as for enforcing the ABAC policies. We release two versions of the ABAC enhanced MySQL code - one that does not use environmental attributes and one with full support. Our experimental results indicate that supporting ABAC using the proposed approach does not add significant overhead during query execution. Making ABAC support possible for databases gives more power over restricting access to various database objects.

As future work, we plan to develop a generic pluggable module for ASQLPLUS which can be deployed in any relational database. One other direction is to further increase the efficiency of PolTree by dynamically updating the nodes of the tree based on the incoming access request patterns. We would also like to install AEMS on a server class machine and carry out experiments with larger attribute sizes, more concurrent operations as well as higher number of rows in the tables.

REFERENCES

- [1] E. Bertino, G. Ghinita, and A. Kamra, "Access control for databases: Concepts and systems," *Foundations and Trends in Databases*, vol. 3, no. 1–2, pp. 1–148, 2011.
- [2] P. Colombo and E. Ferrari, "Towards a unifying attribute based access control approach for NoSQL datastores," in *2017 IEEE 33rd International Conference on Data Engineering (ICDE)*, 2017, pp. 709–720.
- [3] E. Gupta, S. Sural, J. Vaidya, and V. Atluri, "Enabling attribute-based access control in NoSQL databases," *IEEE Transactions on Emerging Topics in Computing*, pp. 1–15, 2022.
- [4] P. Paul, T. Gupta, and S. Sural, "Poster: ASQL - attribute based access control extension for SQL," in *Proceedings of the 27th ACM Symposium on Access Control Models and Technologies*, 2022, pp. 259–261.
- [5] R. Nath, S. Das, S. Sural, J. Vaidya, and V. Atluri, "PolTree: A data structure for making efficient access decisions in ABAC," in *Proceedings of the 24th ACM Symposium on Access Control Models and Technologies*, 2019, pp. 25–35.
- [6] G. Meena. (2023) Github link. [Online]. Available: https://github.com/inifnate/mysql-server/tree/abac_v2_binary_tree
- [7] P. P. Griffiths and B. W. Wade, "An authorization mechanism for a relational database system," *ACM Trans. Database Syst.*, vol. 1, no. 3, pp. 242–255, Sep 1976.
- [8] E. Bertino, P. Samarati, and S. Jajodia, "Authorizations in relational database management systems," in *Proceedings of the 1st ACM Conference on Computer and Communications Security*, 1993, pp. 130–139.
- [9] L. E. Olson, C. A. Gunter, and P. Madhusudan, "A formal framework for reflective database access control policies," in *Proceedings of the 15th ACM Conference on Computer and Communications Security*, 2008, pp. 289–298.
- [10] R. S. Sandhu, "Role-based access control," in *Advances in Computers*. Elsevier, 1998, vol. 46, pp. 237–286.
- [11] S. L. Osborn, "Database security integration using role-based access control," in *Proceedings of the IFIP WG11.3 Annual Working Conference on Database Security*, 2000, pp. 245–257.
- [12] S. Ranise and A. Truong, "Incremental analysis of evolving administrative role based access control policies," in *Proceedings of the IFIP WG11.3 Annual Working Conference on Data and Application Security and Privacy*, 2014, pp. 260–275.
- [13] Y. Shalabi and E. Gudes, "Cryptographically enforced role-based access control for nosql distributed databases," in *Proceedings of the IFIP WG11.3 Annual Working Conference on Database Security*, 2017, pp. 3–19.
- [14] V. C. Hu, D. R. Kuhn, D. F. Ferraiolo, and J. Voas, "Attribute-based access control," *Computer*, vol. 48, no. 2, pp. 85–88, 2015.
- [15] H. Ferraiolo and D. Kuhn, "A comparison of attribute based access control (ABAC) standards for data service applications," National Institute of Standards and Technology, Tech. Rep. 800, 2016.
- [16] D. Ferraiolo, S. Gavrila, G. Katwala, and J. Roberts, "Imposing fine-grain next generation access control over database queries," in *Proceedings of the 2nd ACM Workshop on Attribute-Based Access Control*, 2017, pp. 9–15.
- [17] J.-W. Byun and N. Li, "Purpose based access control for privacy protection in relational database systems," *The VLDB Journal*, vol. 17, no. 4, pp. 603–619, Jul 2008.
- [18] P. Colombo and E. Ferrari, "Enforcement of purpose based access control within relational database management systems," *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 11, pp. 2703–2716, 2014.
- [19] C. Kato, K. Onoue, and Y. Noma, "Application based access control for remote RDBMS," *Journal of Information Processing*, vol. 30, pp. 822–828, 2022.
- [20] S. Jahid, C. A. Gunter, I. Hoque, and H. Okhravi, "MyABDAC: Compiling XACML policies for attribute-based database access control," in *Proceedings of the First ACM Conference on Data and Application Security and Privacy*, 2011, pp. 97–108.
- [21] L. Huang, Y. Zhu, X. Wang, and F. Khurshid, "An attribute-based fine-grained access control mechanism for HBase," in *Proceedings of the International Conference on Database and Expert Systems Applications*, 2019, pp. 44–59.
- [22] R. Ahad, J. Davis, S. Gower, P. Lyngbæk, A. Marynowski, and E. Onuegbue, "Supporting access control in an object-oriented database language," in *Proceedings of the International Conference on Extending Database Technology: Advances in Database Technology*, 1992, pp. 184–200.
- [23] M. Hemdi and R. Deters, "Using REST based protocol to enable ABAC within IoT systems," in *Proceedings of the IEEE Annual Information Technology, Electronics and Mobile Communication Conference*, 2016, pp. 1–7.
- [24] Z. Liu, Z. L. Jiang, X. Wang, S. Yiu, C. Zhang, and X. Zhao, "Dynamic attribute-based access control in cloud storage systems," in *2016 IEEE Trustcom/BigDataSE/ISPA*, 2016, pp. 129–137.
- [25] K. Yang, X. Jia, and K. Ren, "Attribute-based fine-grained access control with efficient revocation in cloud storage systems," in *Proceedings of the 8th ACM Symposium on Information, Computer and Communications Security*, 2013, pp. 523–528.
- [26] A. Swissa and E. Gudes, "An ABAC model with trust and gossiping (ABAC-TG) for online social networks," in *Proceedings of the IFIP WG11.3 Annual Working Conference on Data and Application Security and Privacy*, 2021, pp. 377–392.
- [27] R. Bobba, O. Fatemeh, F. Khan, C. A. Gunter, and H. Khurana, "Using attribute-based access control to enable attribute-based messaging," in *Proceedings of the 22nd Annual Computer Security Applications Conference*, 2006, pp. 403–413.